

# Block 5 – SDN & Cloud-Native Networking

## SDN, NFV, Overlays & Network Automation

Arquitectura de Xarxes

Universitat Pompeu Fabra



- 1 Limitations of Traditional Networks
- 2 Software-Defined Networking (SDN)
- 3 Network Function Virtualization (NFV)
- 4 Cloud-Native Applications
- 5 From VLANs to Overlay Networks
- 6 Container Networking
- 7 Infrastructure as Code & Network Automation
- 8 Session Summary

# Outline

- 1 **Limitations of Traditional Networks**
- 2 Software-Defined Networking (SDN)
- 3 Network Function Virtualization (NFV)
- 4 Cloud-Native Applications
- 5 From VLANs to Overlay Networks
- 6 Container Networking
- 7 Infrastructure as Code & Network Automation
- 8 Session Summary

# What If You Could Program Your Entire Network?

*How are modern networks built and automated under the hood?*

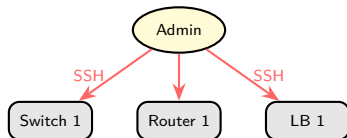
*Traditional networks are configured device by device.  
What if the network was as programmable as software?*

## Learning objectives:

- 1 Explain SDN architecture and why centralized control matters
- 2 Understand NFV and how it replaces hardware appliances
- 3 Describe overlay networks (VXLAN) and why they scale beyond VLANs
- 4 Explain how containers communicate and how Infrastructure as Code automates network management

# The Problem with Traditional Networks

Small network

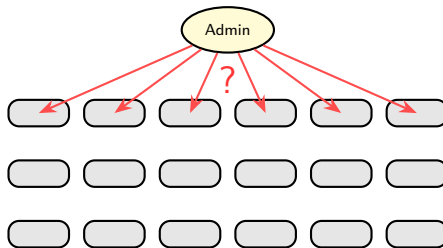


Tedious, but doable

Data center  
grows...



Cloud-scale network



Hundreds of thousands of devices

Days or weeks per change, errors inevitable

At cloud scale, manual configuration breaks completely. There has to be a better way.

## Discussion: The Cost of Manual Networks

*Why have traditional networks survived so long despite their scalability problems?*

- Hint: think about reliability, vendor relationships, and risk aversion
- What would it take for an organization to change?

# Outline

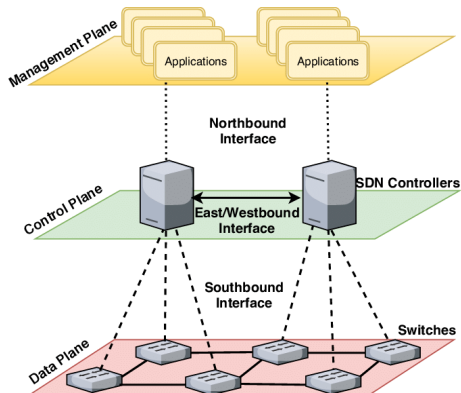
- 1 Limitations of Traditional Networks
- 2 Software-Defined Networking (SDN)**
- 3 Network Function Virtualization (NFV)
- 4 Cloud-Native Applications
- 5 From VLANs to Overlay Networks
- 6 Container Networking
- 7 Infrastructure as Code & Network Automation
- 8 Session Summary

# SDN: The Centralized Brain

## Control plane

The logic inside a network device that decides **where to send traffic**: computing routes, building forwarding tables, reacting to failures. In traditional networks, every device runs its own — SDN extracts it into one central place.

- The controller is the **brain** of the network: global view, all forwarding decisions
- Maintains a real-time **topology database**
- **Programmable via API**: any script or app can read topology, push rules, or automate changes without touching the CLI
  - Uses a **REST API**: the same way your browser loads Google Maps or



# How Does the Controller Talk to Switches?

The controller needs a **common language** to program any switch, regardless of vendor. That language is called the **southbound protocol**.

- **OpenFlow** (Stanford, 2008): the first standard protocol <sup>1</sup>
  - Controller sends **flow rules**: “if packet matches X, do Y”
  - Proved the concept works on real hardware
  - Rarely used in production today
- **Modern alternatives**: gNMI, NETCONF/YANG, vendor APIs (Cisco ACI, VMware NSX)

*Think of it as a universal remote control: one controller, any switch brand.*

<sup>1</sup> ONF, “Software-Defined Networking: The New Norm for Networks,” ONF White Paper, 2012.

# SDN Today: It Is Everywhere

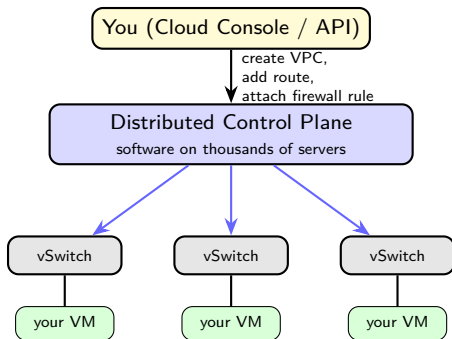
Every time you create a network in AWS, Azure, or GCP, **SDN is doing the work**. No engineer is SSHing into a switch.

- **Cloud providers** (AWS, Azure, GCP): SDN logic distributed across every server in the data center
- **Telcos**: network functions running as **containers**, controlled via software APIs

The SDN **principles** are identical to what we studied:

- Separate decisions from forwarding
- Control via software and APIs

The difference: instead of one controller box, the logic is **distributed across thousands of servers**.



No engineer touches a physical switch

## Discussion: SDN Trade-offs

*If the SDN controller fails,  
what happens to the network?*

- Hint: single point of failure vs distributed control
- How do real deployments address this? (clustering, failover)

# Outline

- 1 Limitations of Traditional Networks
- 2 Software-Defined Networking (SDN)
- 3 Network Function Virtualization (NFV)**
- 4 Cloud-Native Applications
- 5 From VLANs to Overlay Networks
- 6 Container Networking
- 7 Infrastructure as Code & Network Automation
- 8 Session Summary

# Network Functions Virtualization (NFV)

- SDN solves *how* traffic flows: centralized controller programs forwarding rules
- But traffic still hits **dedicated hardware boxes**: firewalls, load balancers, routers
  - Expensive (€10K–€100K+), slow to procure, impossible to scale instantly
- SDN can steer traffic perfectly, but if it leads to a €50K appliance, flexibility is lost
- **NFV**: run those network functions as **software (VNFs)** on commodity servers or VMs <sup>1</sup>

| Physical Appliance     | VNF Equivalent    |
|------------------------|-------------------|
| Hardware firewall      | pfSense, iptables |
| Hardware load balancer | HAProxy, NGINX    |
| Hardware router        | VyOS, FRRouting   |
| IDS/IPS                | Snort, Suricata   |

SDN **steers** traffic. NFV **processes** it. Together they replace the traditional hardware stack.

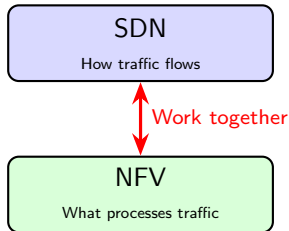
<sup>1</sup> ETSI GS NFV 002, "NFV Architectural Framework," v1.2.1, 2014.

# NFV + Cloud Integration

- Deploy VNFs on cloud infrastructure (IaaS) → **minutes** instead of months
- **Scale horizontally**: add more instances under load
- **Update easily**: software upgrades, no truck rolls
- **Cost reduction**: commodity servers vs specialized hardware

No hardware procurement, no shipping, no rack-and-stack.

# SDN + NFV: Complementary Technologies



## Example: HTTP request entering a data center

- ❶ SDN controller sees incoming traffic
  - ❷ Forwards it to a **VNF firewall** (NFV) for inspection
  - ❸ Firewall approves; SDN routes it to the **VNF load balancer**
  - ❹ Load balancer distributes to backend servers
- SDN **steers** traffic (forwarding rules)
  - NFV **processes** traffic (filter, balance, inspect)

## Discussion: When Hardware Still Wins

*Can you think of a scenario where a physical appliance is better than a VNF?*

- Hint: think about latency, throughput, and specialized workloads
- What about hardware accelerators (FPGAs, SmartNICs)?

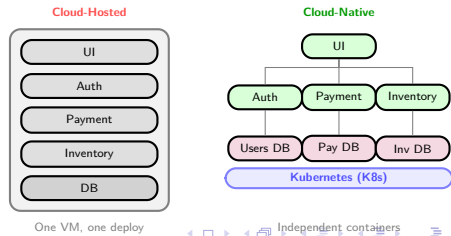
# Outline

- 1 Limitations of Traditional Networks
- 2 Software-Defined Networking (SDN)
- 3 Network Function Virtualization (NFV)
- 4 Cloud-Native Applications**
- 5 From VLANs to Overlay Networks
- 6 Container Networking
- 7 Infrastructure as Code & Network Automation
- 8 Session Summary

# What Does Cloud-Native Mean?

- SDN and NFV give us programmable, virtualized infrastructure
- But applications can still be designed the **old way**: one big monolith, manually deployed, that happens to run in the cloud
- **Cloud-native** is the design philosophy that fully exploits what SDN, NFV, and containers make possible
- Key question: are you just *lifting and shifting* an old app, or designing for the cloud from the start?

|         | Cloud-Hosted      | Cloud-Native      |
|---------|-------------------|-------------------|
| Design  | Monolith in a VM  | Microservices     |
| Scaling | Manual / vertical | Auto / horizontal |
| Failure | Restart whole app | Isolate service   |
| Deploy  | Manual steps      | CI/CD pipeline    |



# Cloud-Native Design Principles

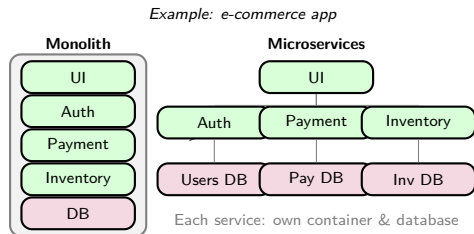
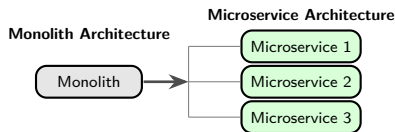
Cloud-native applications are built around a small set of principles: <sup>1</sup>

- **Microservices:** one responsibility per service; independent deploy and scale
- **Containers:** standard, portable packaging; same image from laptop to production
- **Declarative configuration:** describe *what* you want, not *how* to achieve it
- **Immutable infrastructure:** never patch a running instance; replace it with a new image
- **Automated delivery:** CI/CD pipelines build, test, and deploy without manual steps

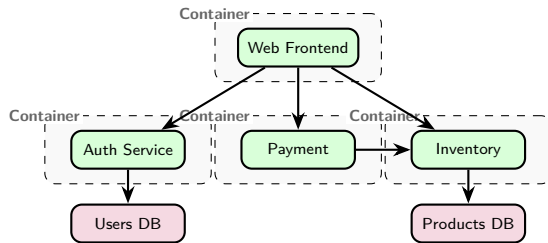
<sup>1</sup> Burns et al., *Kubernetes: Up and Running*, 3rd ed., O'Reilly, 2022.

# Microservices: From Monolith to Distributed

- **Monolithic** app: one big process, one deployment, one codebase
- **Microservices**: app split into small, independent services, each in its own container
- Rule of thumb: **1 microservice = 1 container**



# Microservices: Network Challenges



- Services communicate over the **network** (HTTP/REST, gRPC)
- Network implications for hundreds of containers:
  - **Service discovery**: "where is the payment service?"
  - **Load balancing**: distribute requests across replicas
  - **Observability**: trace requests across services
- Every arrow = **network call** → latency and failure risk
- If Inventory is down → Payment and Web are affected (**cascading failure**)
- Solutions: retries, timeouts, circuit breakers, service meshes

# Kubernetes: The Cloud-Native Platform

These principles require a platform that handles scheduling, networking, scaling, and self-healing automatically. That platform is **Kubernetes (K8s)**, introduced in Block 4 Session 2.

From a networking perspective, K8s provides:

- Each **Pod** gets its own IP address (dynamic, managed by the platform)
- **Services** provide stable DNS names regardless of which host Pods run on
- **Ingress** controllers expose services externally with L7 routing
- **Network Policies** define which Pods can talk to which (micro-segmentation)
- **Container Network Interface (CNI)** plugins handle the overlay wiring (typically VXLAN-based)

Cloud-native networking requirements (dynamic IPs, cross-host comms, load balancing, policies) are exactly what K8s was built to solve.



## Discussion: Is Cloud-Native Always Better?

*A bank has a 15-year-old monolithic payment application running on two dedicated servers.*

*Should they rewrite it as cloud-native microservices?*

- Hint: think about cost, risk, team skills, and whether the current system actually has a scaling problem
- Is “cloud-native” a tool for specific problems, or always the right answer?

# Outline

- 1 Limitations of Traditional Networks
- 2 Software-Defined Networking (SDN)
- 3 Network Function Virtualization (NFV)
- 4 Cloud-Native Applications
- 5 From VLANs to Overlay Networks**
- 6 Container Networking
- 7 Infrastructure as Code & Network Automation
- 8 Session Summary

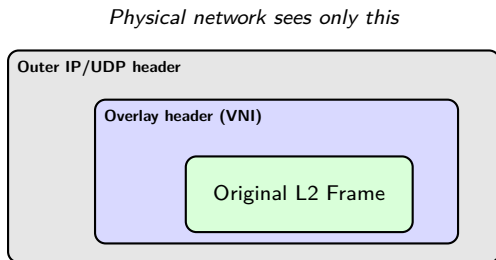
# Why Overlay Networks?

- SDN and NFV give us programmable, virtualized infrastructure
- Multiple tenants share the same physical network: their traffic must be **isolated**
- We know VLANs (Block 3), but VLANs have hard limits:
  - **12-bit ID**: maximum 4,096 networks
  - **Layer 2 only**: cannot span Layer 3 boundaries
  - Moving a VM to another host may require VLAN reconfiguration
- A cloud provider hosting millions of tenants cannot rely on VLANs
- **Overlay networks** solve this: scalable isolation that works across any physical topology

We need a technology that scales beyond 4K networks and works across L3 boundaries.

# Overlay Networks: Concept

- An **overlay network** is a virtual network built **on top of** the physical one
- Uses **encapsulation**: the original frame is wrapped inside a physical packet
- The physical network only sees the **outer headers**: tenant traffic is invisible
- Each tenant gets **full isolation** without the physical network knowing about them



# VXLAN: Overview

- **VXLAN** (Virtual eXtensible LAN): most widely used overlay protocol
- Encapsulates L2 frames in **UDP packets** over the physical network
- Uses a **24-bit VNI** (VXLAN Network Identifier)
  - $2^{24} = \mathbf{16 \text{ million}}$  virtual networks (vs 4,096 VLANs)
- Endpoints: **VTEPs** (VXLAN Tunnel Endpoints) <sup>1</sup>
  - Encapsulate at source, decapsulate at destination

<sup>1</sup> IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.

# VXLAN: How It Works



- VM A sends a frame → VTEP 1 wraps it in UDP with a VNI
- Travels over the physical network as a regular UDP packet
- VTEP 2 strips outer headers, delivers original frame to VM B
- VMs on the **same VNI** form an isolated L2 domain

# Overlay Benefits for Scalability

- **16 million virtual networks** (vs 4,096 VLANs)
- Physical network does not need to know about tenants
- VMs can **migrate** across hosts without reconfiguring the underlay
- Decouples **virtual topology** from physical topology <sup>1</sup>

<sup>1</sup> IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.

## Discussion: Overlay Networks

*Why can the physical network remain simple  
if we use overlay networks?*

*What are the trade-offs of encapsulation?*

Hints: think about overhead (extra headers), Maximum Transmission Unit (MTU) implications, and troubleshooting complexity.

# Outline

- 1 Limitations of Traditional Networks
- 2 Software-Defined Networking (SDN)
- 3 Network Function Virtualization (NFV)
- 4 Cloud-Native Applications
- 5 From VLANs to Overlay Networks
- 6 Container Networking**
- 7 Infrastructure as Code & Network Automation
- 8 Session Summary

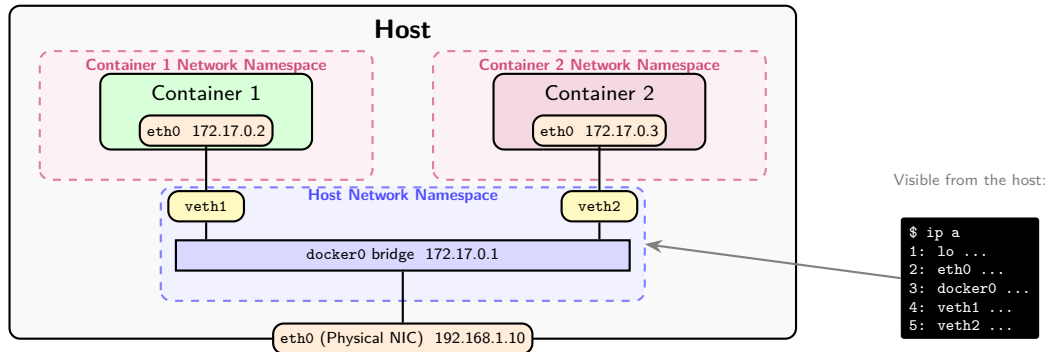
# Why Container Networking?

- Overlay networks give us scalable, isolated virtual networks across any physical infrastructure
- Modern applications are split into **dozens of independent services**, each deployed as a container
- Each container needs its own **network identity** (IP address, hostname)
- Containers must reach each other, possibly on **different physical hosts**
- The overlay networks we just studied are exactly what makes cross-host container communication possible at scale

How does networking work inside and between container hosts?

# Container Networking: Starting Point

- Every container gets its own **network namespace**: isolated IP stack, interfaces, routing table
  - Network namespace: private isolated environment where the container sees only its own interfaces and routes
- Docker wires containers via **virtual bridges** and **veth pairs** (virtual cable)
- Containers reach each other, the host, and the internet depending on the **network model**



# Container Network Models

- **Bridge network (default):**

- Containers on the same host connected via a **virtual bridge** (docker0) <sup>1</sup>
- Each container gets a **veth pair** (virtual Ethernet interface)
- **NAT** for external traffic; containers share host's IP

- **Host network:**

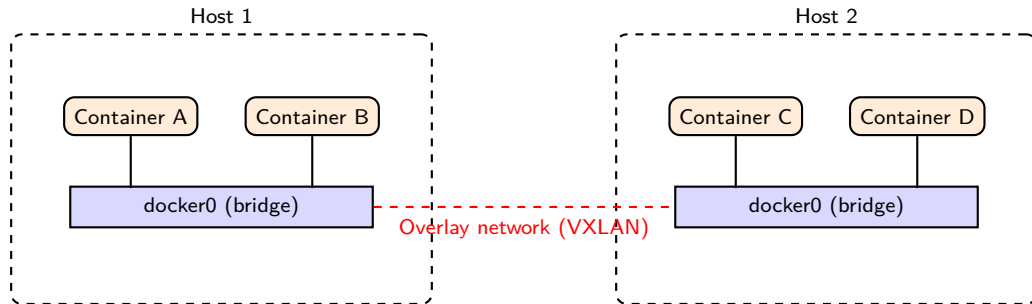
- Container uses the **host's network stack** directly
- No network isolation, but **no NAT overhead**
- Use for performance-critical applications

- **Overlay network:**

- Containers on **different hosts** connected via VXLAN overlay
- Same overlay concept we just covered, now applied to containers
- Enables **multi-host** container deployments

<sup>1</sup> Docker, Inc., "Docker Networking Documentation," docs.docker.com/network

# Container Networking: Visual



- Within a host: containers use a **bridge** network
- Across hosts: an **overlay** connects the bridges (VXLAN from the previous section)

## Discussion: Containers vs VMs

*If containers are faster and lighter than VMs,  
why do companies still use VMs?*

- Hint: security isolation, legacy applications, compliance requirements
- In practice, containers often run inside VMs

# Outline

- 1 Limitations of Traditional Networks
- 2 Software-Defined Networking (SDN)
- 3 Network Function Virtualization (NFV)
- 4 Cloud-Native Applications
- 5 From VLANs to Overlay Networks
- 6 Container Networking
- 7 Infrastructure as Code & Network Automation**
- 8 Session Summary

# Infrastructure as Code (IaC)

- SDN, NFV, overlays, and cloud-native containers are all programmable — but if still configured **by hand**, we are back to slow, error-prone, and unreproducible
- **IaC**: manage infrastructure through **code files**, not manual clicks or dashboards
- Describe the desired state in a file → tools apply it automatically
- **Reproducibility**: same config → same infrastructure, every time
- **Version control**: track changes in Git, review in pull requests, rollback instantly
- **Speed**: deploy entire environments in minutes <sup>1</sup>

<sup>1</sup> Morris, \*Infrastructure as Code\*, 2nd ed., O'Reilly, 2021.

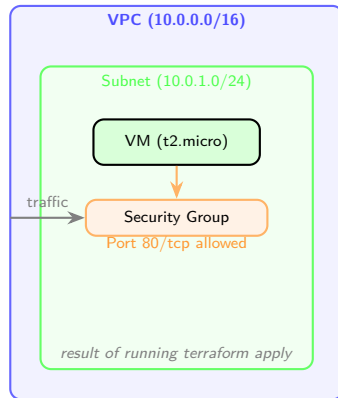
# IaC Example: Terraform (Conceptual)

```
# Define a VPC 1
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

# Define a subnet
resource "aws_subnet" "public" {
  vpc_id = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
}

# Define a VM (EC2 instance)
resource "aws_instance" "web" {
  ami = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
  subnet_id = aws_subnet.public.id
}

# Define a security group
resource "aws_security_group" "web" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
  }
}
```



<sup>1</sup> HashiCorp, "Terraform Documentation," [terraform.io/docs](https://www.terraform.io/docs).

## Discussion: IaC Risks

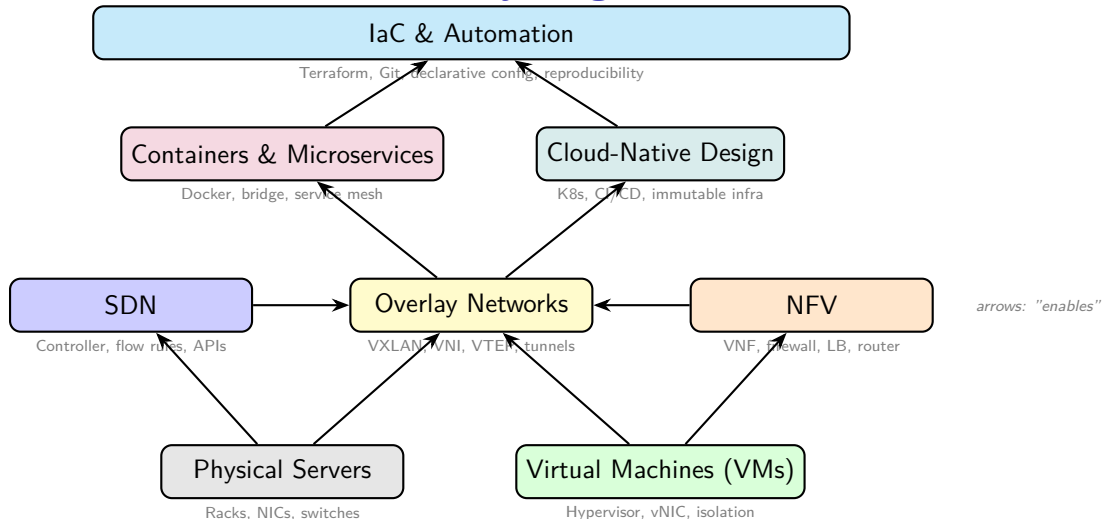
*If infrastructure is defined as code,  
what happens when someone pushes a bug?*

- Hint: code review, staging environments, automated testing
- How is this similar to software development best practices?

# Outline

- 1 Limitations of Traditional Networks
- 2 Software-Defined Networking (SDN)
- 3 Network Function Virtualization (NFV)
- 4 Cloud-Native Applications
- 5 From VLANs to Overlay Networks
- 6 Container Networking
- 7 Infrastructure as Code & Network Automation
- 8 Session Summary**

# The Full Picture: Where Everything Fits



Each layer builds on the previous one: from bare metal to fully automated cloud-native infrastructure.

# Key Takeaways

- 1 Traditional networks are **manual, rigid, and do not scale**
- 2 **SDN** separates control from data plane: centralized, programmable, API-driven
- 3 **NFV** replaces hardware appliances with software: flexible, fast to deploy, cheap
- 4 SDN and NFV are **complementary**: SDN steers traffic, NFV processes it
- 5 **VXLAN** overlays scale isolation to 16 million networks (vs 4,096 VLANs)
- 6 Containers use **bridge networks** on a single host and **overlay networks** across hosts
- 7 **Cloud-native** means designed for the cloud: microservices, containers, declarative config, immutable infra, automated delivery
- 8 **Microservices** require service discovery, load balancing, and resilience mechanisms (circuit breakers)
- 9 **IaC** automates infrastructure: reproducible, version-controlled, auditable

# Discussion

*A company runs 200 microservices in containers.  
One Friday at 5 PM, a manual network change  
breaks communication for 50 of them.  
How could SDN and IaC have prevented this?*

Hints: centralized control, automated testing, instant rollback, reproducibility.

# References

- ❶ McKeown et al., “OpenFlow: Enabling Innovation in Campus Networks,” *ACM SIGCOMM CCR*, vol. 38, no. 2, 2008.
- ❷ ONF, “Software-Defined Networking: The New Norm for Networks,” ONF White Paper, 2012.
- ❸ OpenDaylight Project, [opendaylight.org](http://opendaylight.org).
- ❹ ONOS Project, [onosproject.org](http://onosproject.org).
- ❺ ETSI, “Network Functions Virtualisation,” White Paper, 2012.
- ❻ ETSI GS NFV 002, “NFV Architectural Framework,” v1.2.1, 2014.
- ❼ Docker, Inc., “Docker Networking Documentation,” [docs.docker.com/network](https://docs.docker.com/network).
- ❽ Morris, *Infrastructure as Code*, 2nd ed., O'Reilly, 2021.
- ❾ HashiCorp, “Terraform Documentation,” [terraform.io/docs](https://terraform.io/docs).
- ❿ Goransson & Black, *Software Defined Networks*, 2nd ed., Morgan Kaufmann, 2017.
- ⓫ Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
- ⓬ IETF RFC 7348, “Virtual eXtensible Local Area Network (VXLAN),” 2014.
- ⓭ IETF RFC 8926, “Geneve: Generic Network Virtualization Encapsulation,” 2020.
- ⓮ CNCF, “Cloud Native Definition v1.0,” [github.com/cncf/toc](https://github.com/cncf/toc), 2018.
- ⓯ Burns et al., *Kubernetes: Up and Running*, 3rd ed., O'Reilly, 2022.
- ⓰ Latif et al., “A Comprehensive Survey of Interface Protocols for Software Defined Networks,” arXiv:1902.07913, 2019.